

Operating System

Prepared By:

Er. Ayush Chaudhary

Process Management

Process:

- A process is basically a program in execution.
- A process is defined as an entity that represents the basic unit of work to be implemented in the system.
- We write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.
- When a program is loaded into the memory and it becomes a process, it can be divided into four sections — **stack, heap, text and data**.

Stack

The process Stack contains the temporary data such as method/function parameters, return address and local variables.

Heap

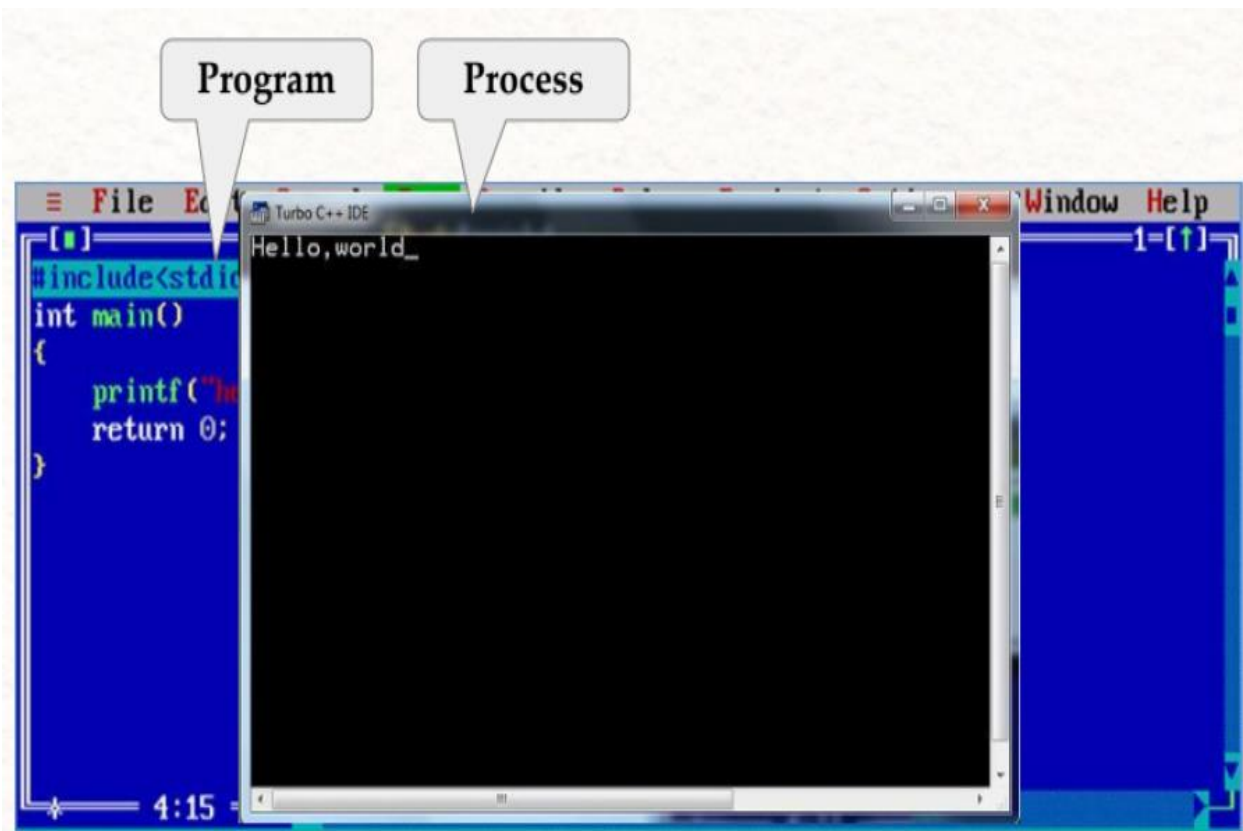
This is dynamically allocated memory to a process during its run time.

Text

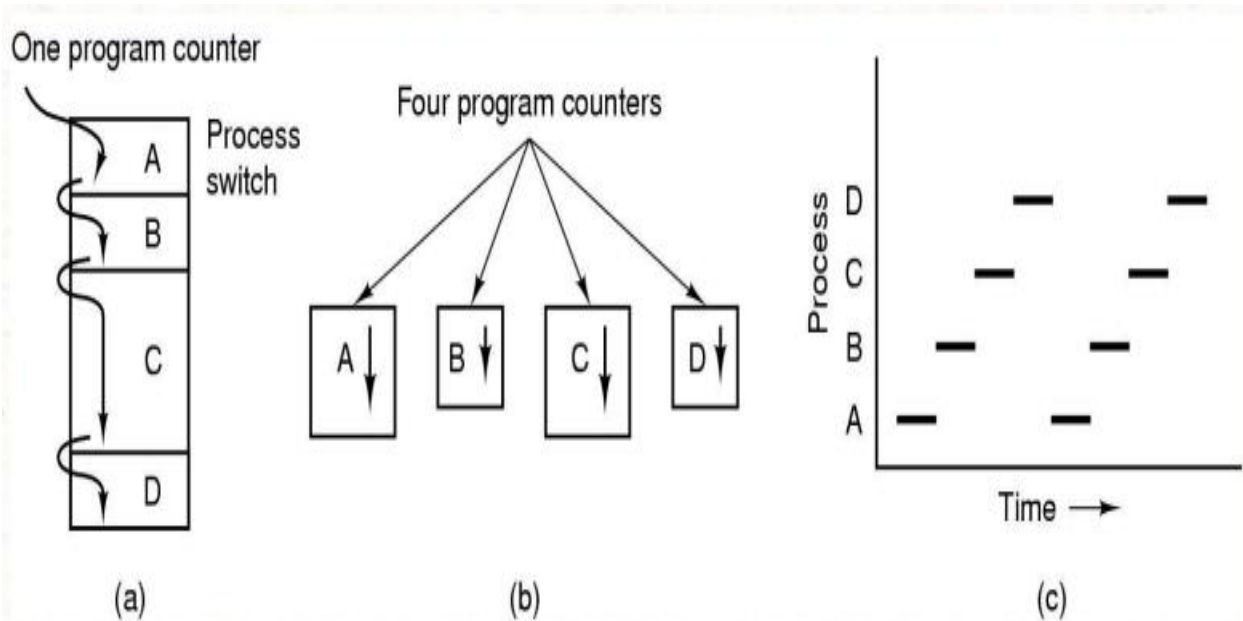
This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.

Data

This section contains the global and static variables.



Process Model:



- Fig. (a) Multiprogramming of **four programs in memory** [Model -1]
- Fig. (b) **Conceptual model of 4 independent, sequential processes**, each with its own flow of control (i.e., its own logical program counter) and each one running independently of the other ones. [Model -2]
- Fig. (c) over a long period of time interval, all the processes have made progress, but **at any given instant only one process is actually running**. [Model -3]

Process State:

- **New** – The process is being created.
- **Running** – In this state the instructions are being executed.
- **Waiting** – The process is in waiting state until an event occurs like I/O operation completion or receiving a signal.
- **Ready** – The process is waiting to be assigned to a processor.
- **Terminated** – the process has finished execution.
- **Suspend ready** – Process that was initially in the ready state but was swapped out of main memory(refer Virtual Memory topic) and placed onto external storage by scheduler is said to be in suspend ready state. The process will transition back to ready state whenever the process is again brought onto the main memory.
- **Suspend wait or suspend blocked** – Similar to suspend ready but uses the process which was performing I/O operation and lack of main memory

caused them to move to secondary memory. When work is finished it may go to suspend ready.

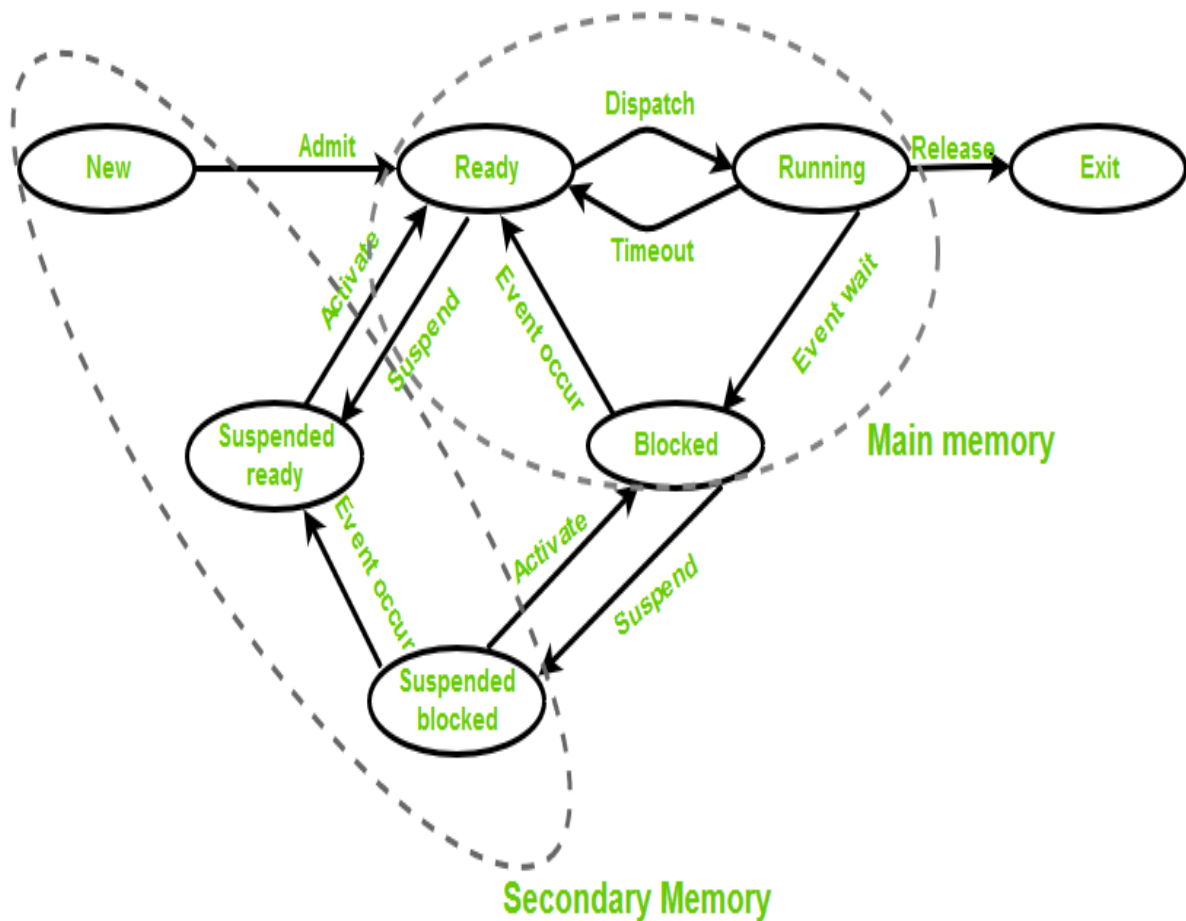
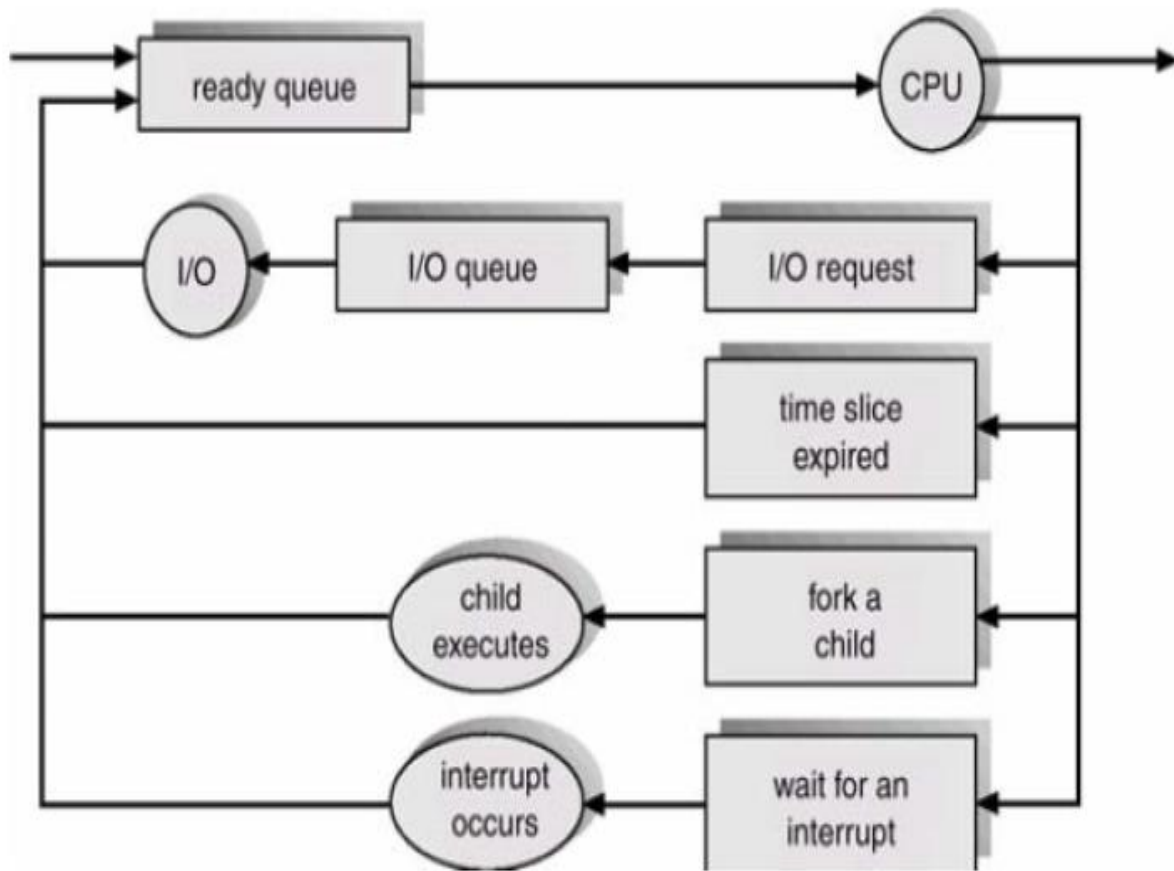


Fig. : Process State Transition

Queue Diagram:



Schedulers:

- **Long-term scheduler (or job scheduler)** –selects which processes should be brought into the ready queue (i.e, selects processes from pool (disk) and loads them into memory for execution).
- **Short-term scheduler (or CPU scheduler)** –selects which process should be executed next and allocates CPU (i.e, selects from among the processes that are ready to execute, and allocates the CPU to one of them) .
- **Short-term scheduler** is invoked very frequently(milliseconds) $\Rightarrow$ (must be fast).
- **Long-term scheduler** is invoked very infrequently(seconds, minutes) $\Rightarrow$ (may be slow).
- **The long-term scheduler** controls the degree of multiprogramming (the number of processes in memory).
- **Medium-term scheduler** –Suspension decision is taken by medium term scheduler. Medium term

scheduler is used for swapping that is moving the process from main memory to secondary and vice versa.

Process Control Block (PCB):

- A Process Control Block is a **data structure maintained by the operating system** for every process.
- PCB keeps all the **information needed to keep track** of a process.
- It is used for **storing the collection of information** about the processes.
- The PCB is maintained for a process throughout its **lifetime** and is **deleted** once the **process terminates**.

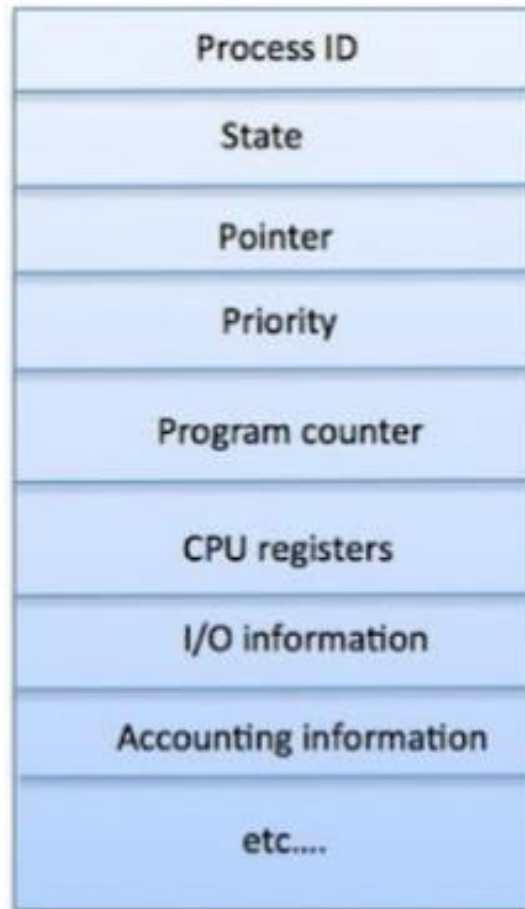
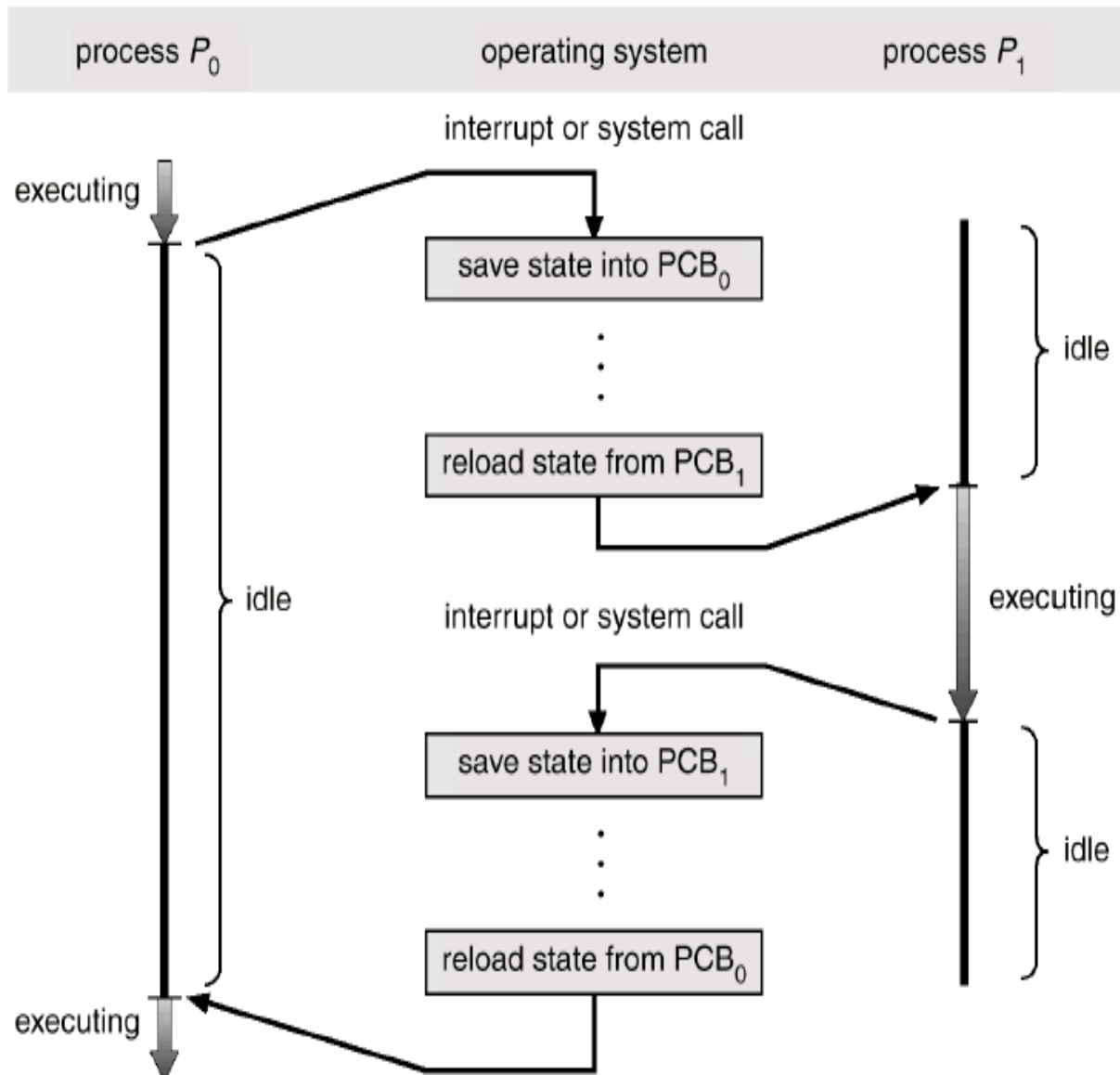


Fig.: Process Control Block

- **Process ID:** Unique identification for each of the process in the operating system.
- **Process State:** The current state of the process i.e., whether it is ready, running, blocked.
- **Pointer:** A pointer to parent process.
- **Priority:** Priority of a process.
- **Program Counter:** Pointer to the address of the next instruction to be executed for this process.

- **CPU registers:** Various CPU registers where process need to be stored for execution for running state.
- **IO status information:** This include a list of I/O devices allocated to the process.
- **Accounting information:** This includes the amount of CPU used for process execution, time limits etc.

CPU Switch Form Process to Process:



Operation on processes:

Process Creation:

There are different operations in the system for which a process can be created. Some of the events that lead to process creation are:

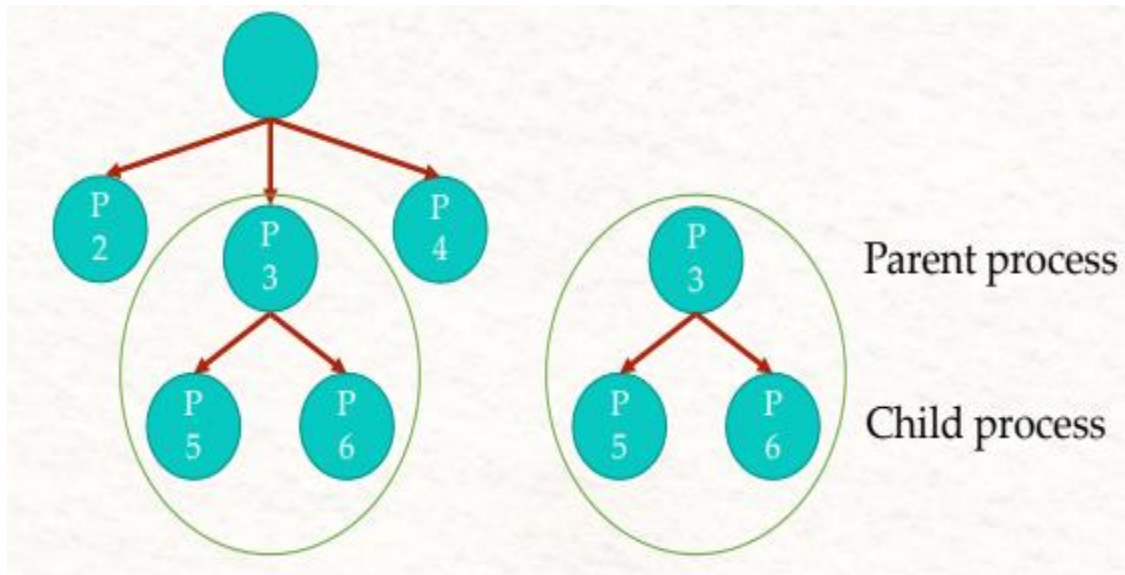
- User requests to create a new process
 - Start process by clicking an icon (opening word file by double click) or by typing command.
- System initialization
 - At the time of system (OS) booting various process are created.
 - Foreground and background processes are created
 - Foreground process- that do not interact with user
 - Background process-that interact with user
- Batch job initialization
- A running process requires the creation of another process using a system call (fork).

Process Termination:

- After the process has completed the execution of its last instruction, it is terminated.
- The resources held by a process are released after it is terminated.
- A child process can be terminated by its parent process if its task is no longer relevant.
- The child process sends its status information to the parent process before it terminates.
- Also, when a parent process is terminated, its child processes are terminated as well as the child processes cannot run if the parent processes are terminated.
- Sometimes the process causes an error due to which it has to terminate e.g **Divide by zero error.**

Process Hierarchies:

- Parent process can create child process, child process can create its own child process.



- Unix has hierarchy concept which is known as process group.
- Windows has no concept of hierarchy.

Process Implementation:

- To implement the process model, the operating system maintains a table (an array of structures), called the process table, with one entry per process.
- This entry contains important information about the process' state, including its program counter, stack pointer, memory allocation, the status of its open files, its accounting and scheduling information, and everything else about the process that must be saved
- The process is switched from running to ready or blocked state so that it can be restarted later as if it had never been stopped.

Cooperating Processes:

Cooperating Processes are those processes that depend on other processes or processes.

Cooperating Process in the operating system is a process that gets affected by other processes under execution or can affect any other process under execution.

They work together to achieve a common task in an operating system.

It shares data with other processes in the system by directly sharing a logical space which is both code and data or by sharing data through files or messages.

IPC(Inter-Process Communication):

- Processes frequently need to communicate with other processes i.e they need to exchange data and information. Thus there is need a need for communication between processes, preferably in a well-structured way not using interrupts. This communication mechanism between processes are known as Inter-Process Communication.

- There are two fundamental models of interprocess communication: **shared memory and message passing**.
- In the shared-memory model, a region of memory that is shared by cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region.
- In the message-passing model, communication takes place by means of messages exchanged between the cooperating processes.

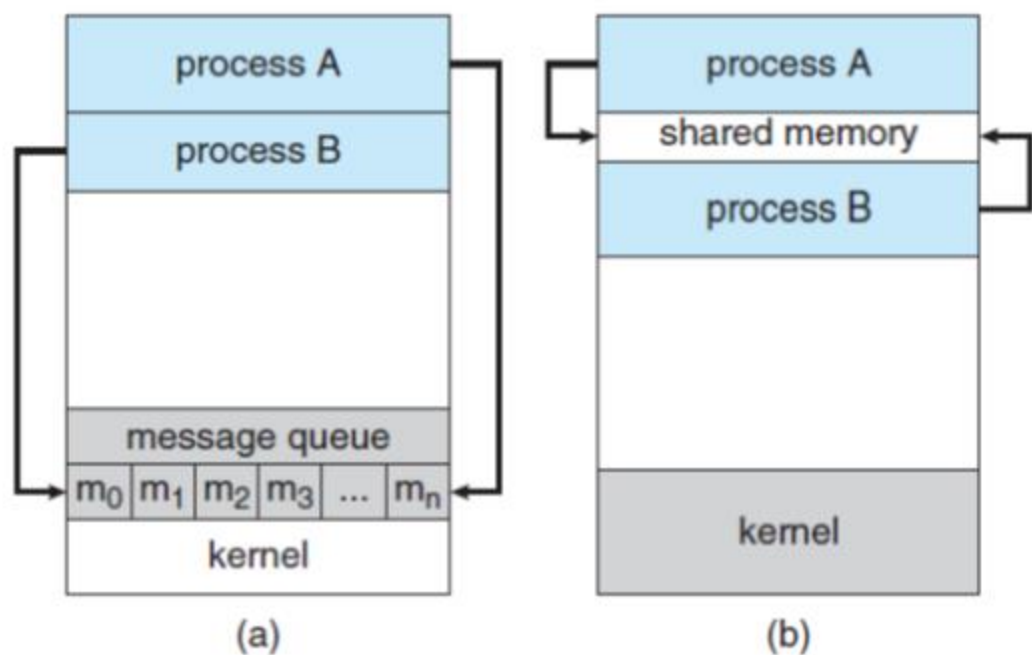


Fig.: a) Message passing b) Shared memory

System Calls:

A system call is a programmatic way in which a computer program requests a service from the kernel of the operating system it is executed on.

A system call is a way for programs to interact with the operating system.

A computer program makes a system call when it makes a request to the operating system's kernel.

System call provides the services of the operating system to the user programs via Application Program Interface(API).

Process Management:

- end, abort°
- load, execute°
- create process, terminate process°
- get process attributes, set process attributes°
- wait for time°
- wait event, signal event°
- allocate and free memory

File management:

- create file, delete file°
- open, close°
- read, write, reposition°
- get file attributes, set file attributes

Examples of Windows and Unix System Calls –

	Windows	Unix
Process Management	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()

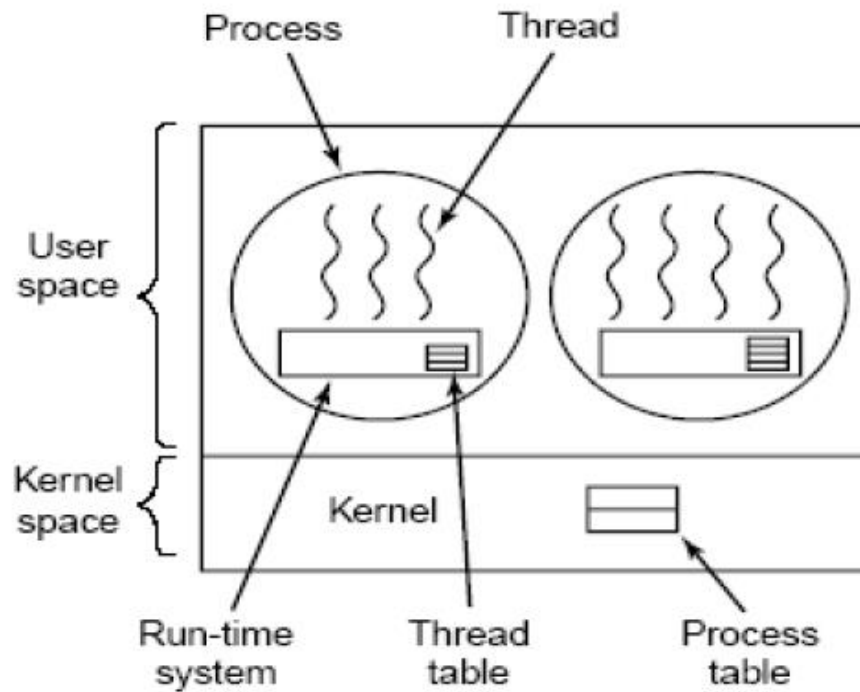
Thread

- A thread is a lightweight process that can run concurrently with other threads within the same process.
- Threads share the same memory space and resources, allowing for more efficient use of resources than creating multiple processes.
- Threads are commonly used in modern operating systems to implement multitasking.
- Threads allow a single process to perform multiple tasks at the same time, which can improve performance and responsiveness.

Types of thread:

User-level threads:

- User threads are supported above the kernel and are implemented by a thread library at the user level.
- The library provides support for thread creation, scheduling, and management with no support from the kernel.
- Because the kernel is unaware of user-level threads, all thread creation and scheduling are done in user space without the need for kernel intervention. Therefore, user-level threads are generally fast to create and manage.



Advantage:

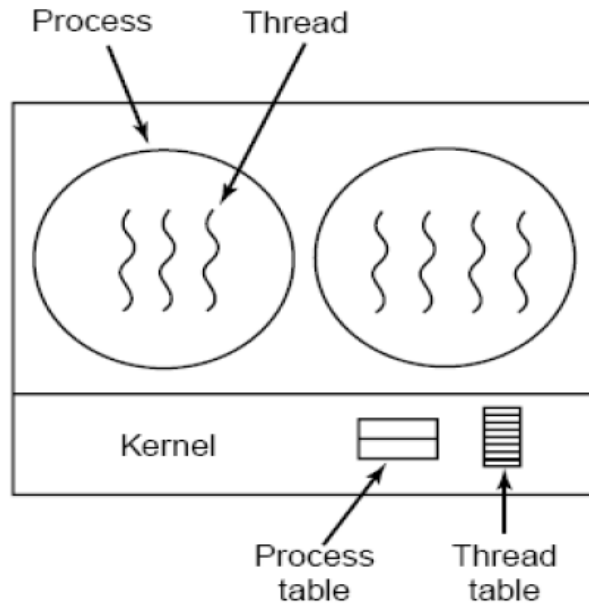
- It is more effective and efficient.
- Context switching takes less time than kernel threads.
- It is more effective and efficient.
- User threads are easier to implement than kernel threads.

Disadvantage:

- Threads at the user level are not coordinated with the kernel.
- The entire operation is halted if a thread creates a page fault.

Kernel-level threads:

- The operating system is recognized by the kernel thread.
- Each thread and process in the kernel-level thread has its own thread control block as well as process control block in the system.
- The operating system implements the kernel-level thread. The kernel is aware of all threads and controls them.
- The kernel-level thread provides a system call for user-space thread creation and management.
- Kernel threads are more complex to build than user threads. The kernel thread's context switch time is longer.



Advantage:

- All threads are completely aware of the kernel-level thread.
- The scheduler may decide to devote extra CPU time to threads with large numerical values.

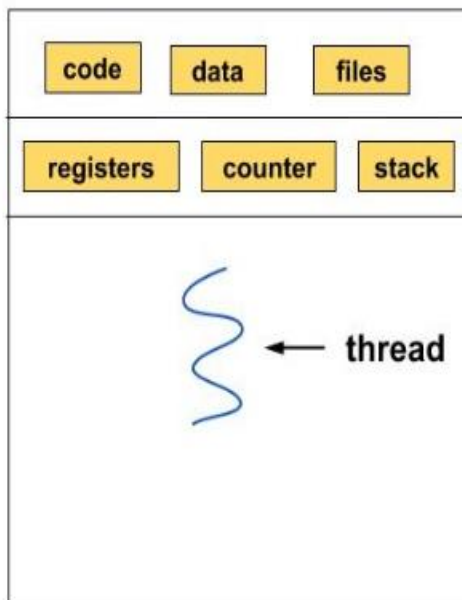
Disadvantage:

- Kernel threads are more complex to build than user threads.
- Kernel-level threads are slower than user-level threads.

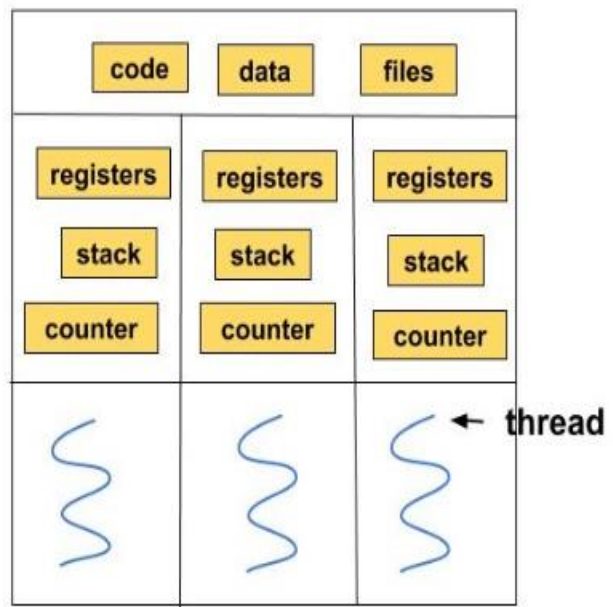
User Level Thread	Kernel Level Thread
User threads are implemented by users.	Kernel threads are implemented by Operating System (OS).
Implementation of User threads is easy.	Implementation of Kernel thread is complicated.
Operating System doesn't recognize user level threads.	Kernel threads are recognized by Operating System
Context switch time is less.	Context switch time is more.
Context switch requires no hardware support.	Hardware support is needed.
Example: Java thread, POSIX threads.	Example: Window Solaris.

Multithreading:

- Multithreading is the ability of an operating System to support multiple threads of execution within a single process.
- Multiple threads run in the same address space, share the same memory areas
 - The creation of a thread only creates a new thread control structure, not a separate process image



Single-threaded process



Multi-threaded process

Multithreading is a technique in operating systems where a single process or program can run multiple threads of execution concurrently.

The **benefits** of multithreading in an operating system are numerous and can include:

1. **Improved Performance:** Multithreading can improve performance by allowing a program to take advantage of available CPU resources more efficiently. By running multiple threads in parallel, a program can execute more tasks simultaneously, thereby reducing the overall processing time.
2. **Increased Responsiveness:** Multithreading can also improve the responsiveness of an operating system by allowing it to handle multiple tasks concurrently. For example, a program can continue to respond to user input while simultaneously performing background tasks.
3. **Resource Sharing:** Multithreading enables multiple threads to share the same resources such as memory, files, and I/O devices, which can lead to more efficient resource utilization.

4. **Scalability**: Multithreading can also improve the scalability of an operating system by allowing it to efficiently utilize multiple processors or CPU cores. This is particularly useful for high-performance computing applications that require significant processing power.
5. **Simplified Programming**: Multithreading can simplify programming by allowing a program to be broken down into smaller, more manageable threads. This can make the program easier to understand, debug, and maintain.

Multithreading Models

Many systems provide support for both user and kernel threads, resulting in different multithreading models.

1. Many-to-One Model:

- All user-level threads of one process are mapped to a single kernel-level thread
- Thread management in user space

–Efficient

–Application can run its own scheduler implementation

- One thread can access the kernel at a time
 - Limited concurrency, limited parallelism

Examples

- Green threads^{||} (e.g. Solaris)
- Gnu Portable Threads

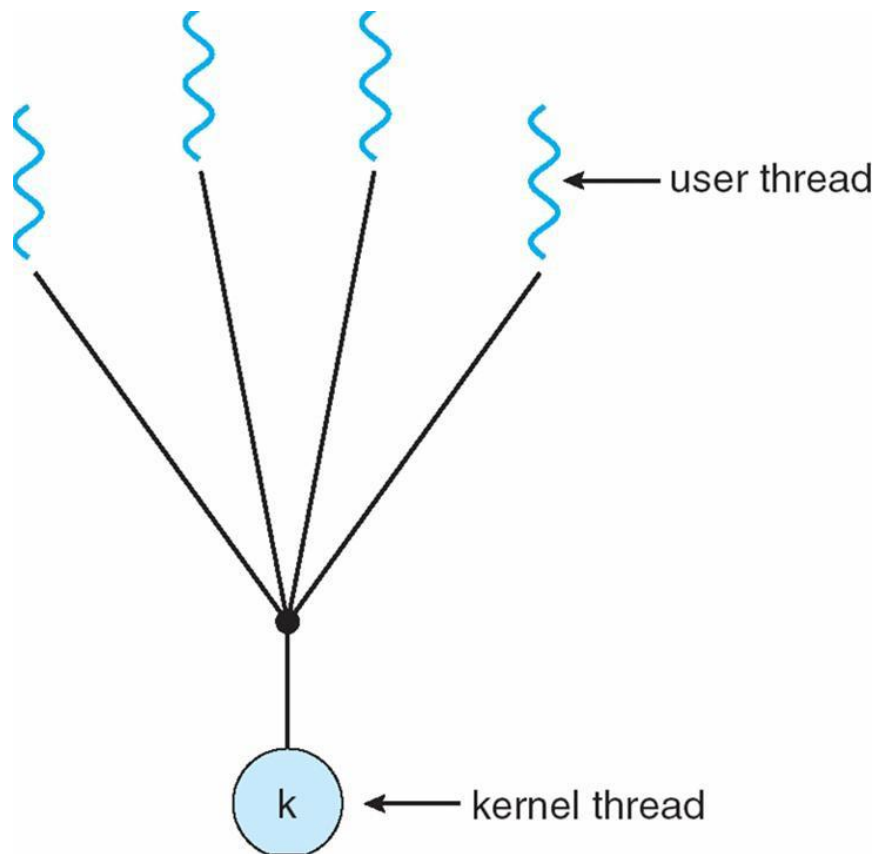


Fig. Many-to-One Model

2. One-to-One Model:

- The one-to-one model maps each user thread to a kernel thread.
- It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call.
- It also allows multiple threads to run in parallel on multiprocessors.
- Examples:- Linux, along with the family of windows operating systems.

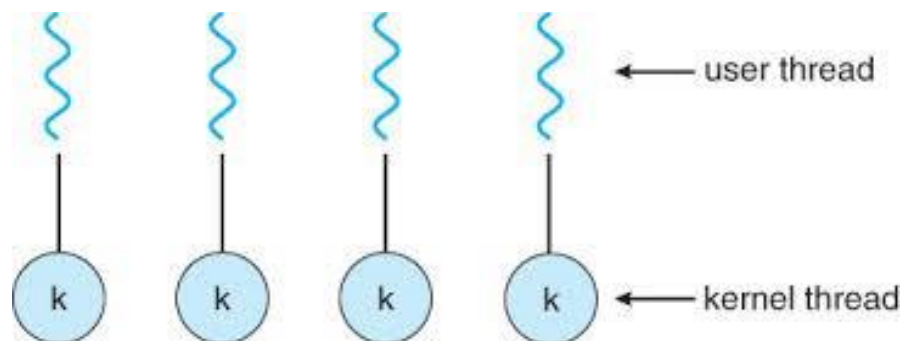


Fig. One-to-One model

3. Many-to-Many Model:

- The many-to-many model multiplexes many user-level threads to a smaller or equal number of kernel threads.
- The number of kernel threads may be specific to either a particular application or a particular machine.
- Also, when a thread performs a blocking system call, the kernel can schedule another thread for execution.

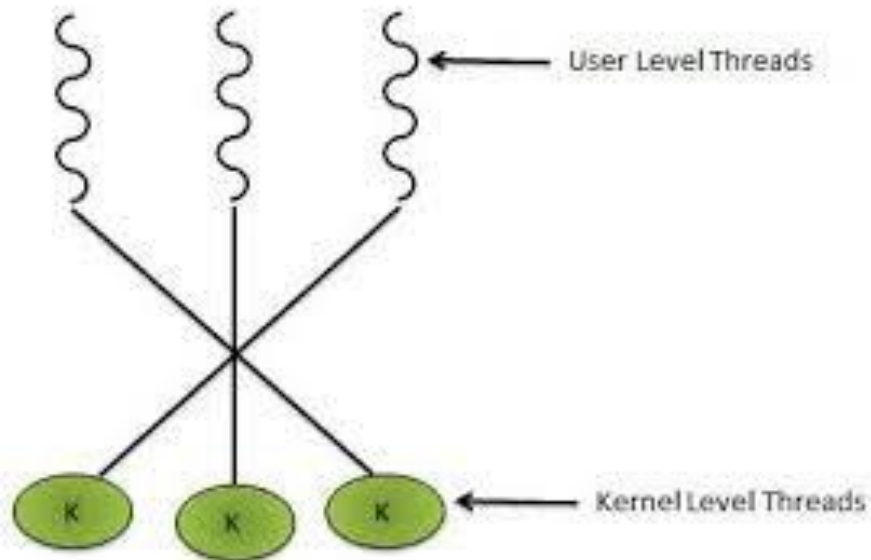


Fig. Many-to-Many Model

Process Synchronization:

Process is categorized into two types on the basis of synchronization and these are given below:

- Independent Process
- Cooperative Process

Independent Processes:

Two processes are said to be independent if the execution of one process does not affect the execution of another process.

Cooperative Processes:

Two processes are said to be cooperative if the execution of one process affects the execution of another process. These processes need to be synchronized so that the order of execution can be guaranteed.

Process synchronization:

- Process synchronization refers to the techniques used to **coordinate the activities of multiple processes or threads** so that they can share resources and communicate with each other in a safe and efficient manner.
- In a multi-tasking operating system, multiple processes may be running at the same time and competing for the same resources such as memory, CPU time, and input/output devices.
- Without proper synchronization, this competition can lead to race condition, deadlocks, and other problems.

Race Condition:

- A race condition is an **undesirable situation** that can occur in multi-threaded or multi-process systems where the behavior of the system depends on the timing of events or the order in which different threads or processes execute.
- In a race condition, two or more threads or processes attempt to access shared resources or variables

concurrently, and the **final outcome** of the program depends on the **order in which these operations are executed**.

- This can result in unpredictable or incorrect behavior, such as data corruption, crashes, or deadlock.

Example:

	P1	P2
	int shared =4;	
	Int x=shared;	int y =shared;
	x++;	y--;
	sleep(1);	sleep(1);
	shared=x;	shared=y;

Result P1 = 5

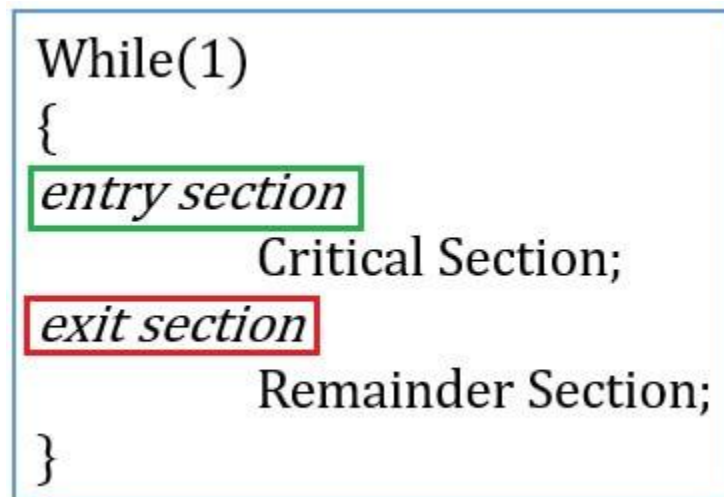
Result P2 = 3

Then Final Result = 3 (Incorrect)

The Actual Result is 4.

Critical Section:

- Critical section refers to a **part of a computer program that must not be executed by more than one process or thread at the same time.**
- This is because the critical section contains **shared resources**, such as variables or data structures, that could be **corrupted or produce incorrect results** if accessed **concurrently**.



Entry Section

In this section mainly the process requests for its entry in the critical section.

Exit Section

This section is followed by the critical section.

The Critical-Section Problem

- n processes all competing to use some shared data .
- Each process has a code segment, called critical section, in which the shared data is accessed.
- **Problem** –ensure that when one process is executing in its critical section, no other process is allowed to execute in its critical section.

The solution to the Critical Section Problem:

A solution to the critical section problem must satisfy the following conditions:

1. **Mutual Exclusion**

Out of a group of cooperating processes, only one process can be in its critical section at a given point of time.

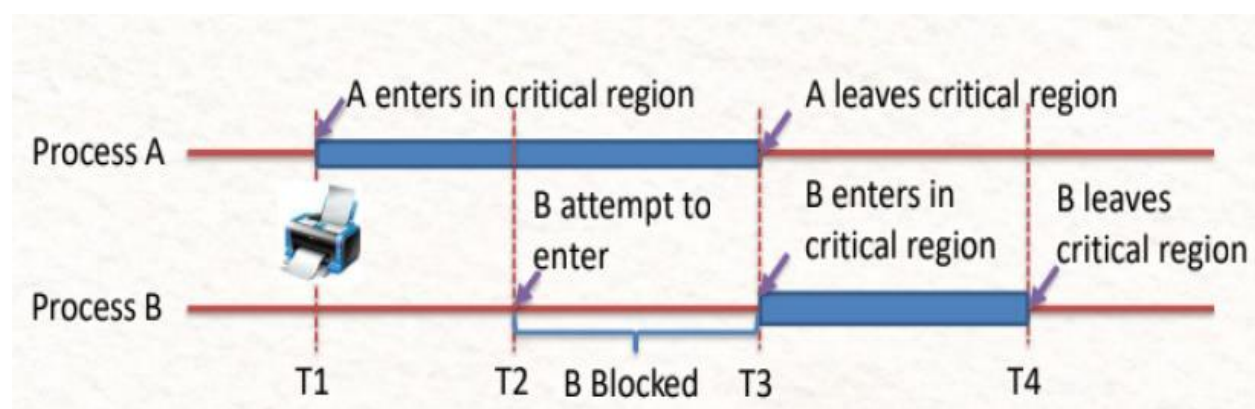


Fig: Mutual Exclusion using Critical Regions

2. **Progress**

If no process is in its critical section, and if one or more threads want to execute their critical section then any one of these threads must be allowed to get into its critical section.

3. **Bounded Waiting**

After a process makes a request for getting into its critical section, there is a limit for how many other processes can get into their critical section, before this process's request is granted.

So after the limit is reached, the system must grant the process permission to get into its critical section.

4. **Arbitrary Speed**

No assumption can be made about the relative speed of different processes.

Proposal for Achieving Mutual Exclusion:

1. Disabling Interrupt

- In a uniprocessor machine, concurrent processes cannot be overlapped; they can only be interleaved.
- Furthermore, a process will continue to run until it invokes an operating system service or until it is interrupted.
- Therefore, to guarantee mutual exclusion, it is sufficient to prevent a process from being interrupted.
- This capability can be provided in the form of primitives defined by the system kernel for disabling and enabling interrupts.
- In this mechanism, each process disables all interrupt just after entering its critical section and re-enables them before leaving it.

eg:

```
while (true) (  
  disable interrupts()  
  critical section  
  enable interrupts()  
) remainder
```

Because the **critical section cannot be interrupted**, **mutual exclusion** is guaranteed.

2. Lock Variables

- A single, shared, (lock) variable, initially 0.
- When a process wants to enter its critical region, it first tests the lock.
- If the lock is 0, the process sets it to 1 and enters the critical region. If the lock is already 1, the process just waits until it becomes 0. Thus, a 0 means that no process is in its critical region, and a 1 means that some process is in its critical region.

// Entry section

```
while (lock != 0);    // Busy-wait until the lock is  
released
```

lock = 1; // Acquire the lock

// Critical section

// Exit section

lock = 0; // Release the lock

3. TSL (Test and Set Lock)

TSL RX, LOCK (Test and Set Lock) that works as follows:

It reads the contents of the memory word LOCK into register RX and then stores a nonzero value at the memory address LOCK. The operations of reading the word and storing into it are guaranteed to be indivisible no other processor can access the memory word until the instruction is finished. The CPU executing the TSL instruction locks the memory bus to prohibit other CPUs from accessing memory until it is done.

Solution:

while (true) {

```
while ( TestAndSet (&lock ))  
; /* do nothing  
// critical section lock = FALSE;  
// remainder section  
}
```

```
boolean TestAndSet (boolean *target)  
{  
boolean rv = *target;  
*target = TRUE;  
return rv;  
}
```

4. Sleep and Wakeup:

Sleep and wakeup are system calls that blocks process instead of wasting CPU time when they are not allowed to enter their critical region.

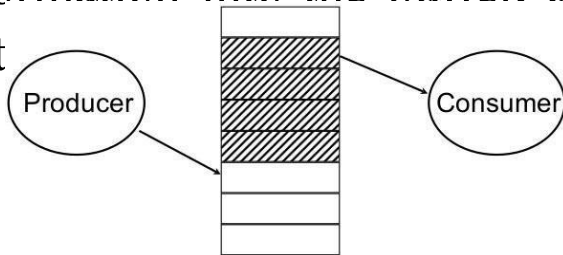
Sleep is a system call that causes the caller to block, that is, be suspended until another process wakes it up. The

wakeup call has one parameter, the process to be awakened.

Examples to use Sleep and Wakeup primitives:

Producer-consumer problem (Bounded Buffer):

Two processes share a common, fixed-size buffer. One of them, the producer, puts information into the buffer. and the other one, the consumer, t



Trouble arises when

1. The producer wants to put a new data in the buffer, but buffer is already full. **Solution:** Producer goes to sleep and to be awakened when the consumer has removed data.
2. The consumer wants to remove data form the buffer but buffer is already empty.

Solution: Consumer goes to sleep until the producer puts some data in buffer and wakes consumer up.

```
#define N 100 /* number of slots in the buffer */
```

```
int count = 0; /* number of items in the buffer */
```

```
void producer(void)
```

```

{
    int item;

    while (TRUE)
    {
        /* repeat forever */
        item = produce_item(); /* generate next item */
        if (count == N)
            sleep(); /* if buffer is full, go to sleep */
        insert_item(item); /* put item in buffer */
        count = count + 1; /* increment count of items in buffer */
        if (count == 1)
            wakeup(consumer); /* was buffer empty? */
    }
}

```

```

void consumer(void)
{
    int item;

    while (true)
    { /* repeat forever */
        if (count == 0)
            sleep(); /* if buffer is empty, got to sleep */
    }
}

```

```

    item = remove_item(); /* take item out of buffer */
    count = count - 1; /* decrement count of items in buffer */
    if (count == N - 1)
        wakeup(producer); /* was buffer full? */
    consume_item(item); /* print item */
}
}

```

N = size of buffer

Count--> a variable to keep track of the no. of items in the buffer.

5. Paterson Solution (Strict Alteration)

This is a software mechanism implemented at user mode. It is a busy waiting solution can be implemented for only two processes. It uses two variables that are turn variable and interested variable.

The Code of the solution is given below

```

# define N 2

# define TRUE 1

```

```

# define FALSE 0
int interested[N] = FALSE;
int turn;
void Entry_Section (int process)
{
    1.int other;
    2.other = 1-process;
    3.interested[process] = TRUE;
    4.turn = process;
    5.while (interested [other] =True && TURN=proc
        ess);
}
void Exit_Section (int process)
{
    interested [process] = FALSE;
}

```

This is a two process solution. Let us consider two cooperative processes P1 and P2. The entry section and exit

section are shown below. Initially, the value of interested variables and turn variable is 0.

Initially process P1 arrives and wants to enter into the critical section. It sets its interested variable to True (instruction line 3) and also sets turn to 1 (line number 4). Since the condition given in line number 5 is completely satisfied by P1 therefore it will enter in the critical section.

1. P1 → 1 2 3 4 5 CS

Meanwhile, Process P1 got preempted and process P2 got scheduled. P2 also wants to enter in the critical section and executes instructions 1, 2, 3 and 4 of entry section. On instruction 5, it got stuck since it doesn't satisfy the condition (value of other interested variable is still true). Therefore it gets into the busy waiting.

1. P2 → 1 2 3 4 5

P1 again got scheduled and finish the critical section by executing the instruction no. 6 (setting interested variable to false). Now if P2 checks then it are going to satisfy the condition since other process's interested variable becomes false. P2 will also get enter the critical section.

1. P1 → 6

2. P2 → 5 CS

Any of the process may enter in the critical section for multiple numbers of times. Hence the procedure occurs in the cyclic order.

Types of the Mutual Exclusion:

1. Semaphore
2. Monitor
3. Mutex
4. Message Passing
5. Bounded Buffer

Semaphore:

Semaphore is an **integer variable** which is used in **mutual exclusive manner** by various concurrent cooperative processes in order to achieve synchronization.

The Semaphore Operations are:

For entry section: P(), Down(), Wait()

For Exit section: V(), Up(), Signal() or Post()

Type of Semaphore:

1. Counting Semaphore
2. Binary Semaphore

Counting Semaphore:

A counting semaphore is a synchronization primitive used in operating systems to control access to a shared resource. It is a variable that can be accessed by multiple threads or processes concurrently, and is used to signal the availability of the resource.

A counting semaphore is initialized with a positive integer value, which represents the number of available resources. When a thread or process wants to use the resource, it first decrements the semaphore value by 1. If the semaphore value is already 0, the thread or process is blocked until the semaphore value becomes positive again.

When the thread or process is done using the resource, it increments the semaphore value by 1, indicating that the resource is available again. If there were other threads or processes waiting for the resource, one of them will be unblocked and granted access.

Implementation of wait:

```
wait (S) {  
value--;  
if (value < 0) {  
add this process to waiting queue block();  
}  
}
```

Implementation of signal:

```
signal (S) {  
value++;  
if (value <= 0) {  
remove a process P from the waiting queue wakeup(P);  
}  
}
```

Binary Semaphore:

a binary semaphore is a type of semaphore that has only two states: 0 and 1. It is also known as a **mutex** (short for "mutual exclusion"), and is typically used to protect a shared resource that can only be accessed by one process or thread at a time.

A binary semaphore works by allowing a process or thread to acquire the semaphore, which sets its value to 1, indicating that the resource is in use. Other processes or threads that attempt to acquire the semaphore while its value is 1 will be blocked until the semaphore is released. When the process or thread is finished using the resource, it releases the semaphore, setting its value back to 0, which allows other processes or threads to acquire the semaphore and access the shared resource.

```
Wait(S)
{
    if (value==1)
    {
        value=0;
    }
}
```

```
    else
    {
        Block this process and place in suspend list sleep();
    }
}
```

Signal(S)

```
{
    if(suspend list is empty);
    {
        Value=1;
    }
    else
    {
        Select a process from suspend list and wakeup();
    }
}
```

Monitors:

Monitors are a synchronization construct that were created to overcome the problems caused by semaphores such as timing errors.

Monitors are abstract data types and contain shared data variables and procedures. The shared data variables cannot be directly accessed by a process and procedures are required to allow a single process to access the shared data variables at a time.

This is demonstrated as follows:

```
monitor monitorName
```

```
{
```

```
    data variables;
```

```
    Procedure P1(...)
```

```
{
```

```
}
```

```
    Procedure P2(...)
```

```
{
```

}

Procedure Pn(...)

{

}

Initialization Code(...)

{

}

}

Only one process can be active in a monitor at a time. Other processes that need to access the shared variables in a monitor have to line up in a queue and are only provided access when the previous process release the shared variables.

Mutexes:

A mutex (short for mutual exclusion) is a synchronization object that is used to protect shared resources from concurrent access by multiple threads or processes.

A mutex allows only one thread or process to access the shared resource at any given time, while all other threads or processes that attempt to access the resource are blocked until the mutex is released.

A mutex is typically implemented as a binary semaphore with an initial value of 1, meaning that it is initially available for use. When a thread or process wants to access the shared resource, it first attempts to acquire the mutex. If the mutex is available (i.e., its value is 1), the thread or process acquires the mutex by setting its value to 0 and proceeds to access the shared resource. If the mutex is not available (i.e., its value is 0), the thread or process is blocked until the mutex becomes available.

Once a thread or process has finished using the shared resource, it releases the mutex by setting its value back to 1, allowing other threads or processes to acquire the mutex and access the shared resource.

A binary semaphore can be used as a Mutex.

Message Passing:

- This method will **use two primitives**
1. Send: It is **used to send message**.
 - Send (destination, &message)
 - In above syntax destination is the process to which sender want to send message and message is what the sender wants to send.
 2. Receive: It is **used to receive message**.
 - Receive (source, &message)
 - In above syntax source is the process that has send message and message is what the sender has sent.

Producer Consumer problem using message passing

```
#define N 100                                //number of slots in buffer
void producer(void)
{
    int item; message m; while
    (true)                                    //message buffer
    {
        item=produce_item();
        receive(consumer, &m);               //generate something to put in buffer
        build_message(&m, item);             //wait for an empty to arrive
        send(consumer, &m);                  //construct a message to send
    }                                         //send item to consumer
}
```

Producer Consumer problem

using message passing

```
void consumer (void)
{
    int item, i; message m;
    for (i=0; i<N; i++) send (producer, &m); //send N empties
    while (true)
    {
        receive (producer, &m); //get message containing item
        item=extract_item(&m); //extract item from message
        send (producer, &m); //send back empty reply
        consume_item (item); //do something with the item
    }
}
```

Classical IPC (Synchronization) Problems

There are various synchronization problems of a large class of concurrency-control problems. These problems are used for testing nearly every newly proposed synchronization scheme.

1. Readers and Writers Problem
2. Dining-Philosophers Problem
3. Sleeping barber Problem

Dining-Philosophers Problem:

There are N philosophers sitting around a circular table eating spaghetti and discussing philosophy. The problem is that each philosopher needs 2 forks to eat, and there are only N forks, one between each 2 philosophers. Design an algorithm that the philosophers can follow that insures that none starves as long as each philosopher eventually stops eating, and such that the maximum number of philosophers can eat at once.

- Philosophers eat/think
- Eating needs 2 forks

- Pick one fork at a time
- How to prevent deadlock



fig: The situation of the dining philosophers

The problem was designed to illustrate the problem of avoiding deadlock, a system state in which no progress is possible.

One idea is to instruct each philosopher to behave as follows:

- Think until the left fork is available; when it is, pick it up
- Think until the right fork is available; when it is, pick it up
- Eat
- Put the left fork down
- Put the right fork down
- Repeat from the start

This solution is incorrect: it allows the system to reach deadlock. Suppose that all five philosophers take their left forks simultaneously. None will be able to take their right forks, and there will be a deadlock.

We could modify the program so that after taking the left fork, the program checks to see if the right fork is available. If it is not, the philosopher puts down the left one, waits for some time, and then repeats the whole process. This proposal too, fails, although for a different reason. With a little bit of bad luck, all the philosophers could start the

algorithm simultaneously, picking up their left forks, seeing that their right forks were not available, putting down their left forks, waiting, and picking up their left forks again simultaneously, and so on, forever. A situation like this, in which all the programs continue to run indefinitely but fail to make any progress is called starvation.

The solution presented below is deadlock-free and allows the maximum parallelism for an arbitrary number of philosophers. It uses an array, state, to keep track of whether a philosopher is eating, thinking, or hungry (trying to acquire forks). A philosopher may move into eating state only if neither neighbor is eating. Philosopher i 's neighbors are defined by the macros LEFT and RIGHT. In other words, if i is 2, LEFT is 1 and RIGHT is 3.

The structure of Philosopher i is shown below:

```
do {  
wait ( chopstick[i] );  
wait ( chopStick[ (i + 1) % 5] );  
... .
```

```
// eat  
  
. . . .  
signal ( chopstick[i] );  
signal (chopstick[ (i + 1) % 5] );  
  
. . .  
// think  
  
. . .  
}  
while (true) ;  
fig: the structure of philosopher i.
```

Readers and Writers Problem:

The dining philosophers' problem is useful for modeling processes that are competing for exclusive access to a limited number of resources, such as I/O devices. Another famous problem is the readers and writers problem which models access to a database (Courtois et al., 1971). Imagine, for example, an airline reservation system, with many competing processes wishing to read and write it. It is acceptable to have multiple processes reading the database at the same time, but if one process is updating (writing) the database, no other process may have access to the database, not even a reader. The question is how do you program the readers and the writers? One solution is shown below.

```
typedef int semaphore; /* use your imagination */
semaphore mutex = 1; /* controls access to 'rc' */
semaphore db = 1; /* controls access to the database */
int rc = 0; /* # of processes reading or wanting to */

void reader(void)
{
  while (TRUE){ /* repeat forever */
    down(&mutex); /* get exclusive access to 'rc' */
    rc = rc + 1; /* one reader more now */
    if (rc == 1) down(&db); /* if this is the first reader ... */
```

```

up(&mutex); /* release exclusive access to 'rc' */
read_data_base(); /* access the data */
down(&mutex); /* get exclusive access to 'rc' */
rc = rc - 1; /* one reader fewer now */
if (rc == 0) up(&db); /* if this is the last reader ... */
up(&mutex); /* release exclusive access to 'rc' */
use_data_read(); /* noncritical region */
}
}
void writer(void)
{
while (TRUE){ /* repeat forever */
think_up_data(); /* noncritical region */
down(&db); /* get exclusive access */
write_data_base(); /* update the data */
up(&db); /* release exclusive access */
}
}

```

In this solution, the first reader to get access to the database does a down on the semaphore *db*. Subsequent readers merely increment a counter, *rc*. As readers leave, they decrement the counter and the last one out does an up on the semaphore, allowing a blocked writer, if there is one, to get in.

Suppose that while a reader is using the database, another reader comes along. Since having two readers at the same time is not a problem, the second reader is admitted. A third

and subsequent readers can also be admitted if they come along.

Now suppose that a writer comes along. The writer cannot be admitted to the database, since writers must have exclusive access, so the writer is suspended. Later, additional readers show up. As long as at least one reader is still active, subsequent readers are admitted. As a consequence of this strategy, as long as there is a steady supply of readers, they will all get in as soon as they arrive. The writer will be kept suspended until no reader is present. If a new reader arrives, say, every 2 seconds, and each reader takes 5 seconds to do its work, the writer will never get in.

To prevent this situation, the program could be written slightly differently: when a reader arrives and a writer is waiting, the reader is suspended behind the writer instead of being admitted immediately. In this way, a writer has to wait for readers that were active when it arrived to finish but does not have to wait for readers that came along after it. The disadvantage of this solution is that it achieves less concurrency and thus lower performance.

The Sleeping Barber Problem

Another classical IPC problem takes place in a barber shop. The barber shop has one barber, one barber chair, and n chairs for waiting customers, if any, to sit on. If there are no customers present, the barber sits down in the barber chair and falls asleep, as illustrated in Fig. When a customer arrives, he has to wake up the sleeping barber. If additional customers arrive while the barber is cutting a customer's hair, they either sit down (if there are empty chairs) or leave the shop (if all chairs are full). The problem is to program the barber and the customers without getting into race conditions. This problem is similar to various queueing situations, such as a multiperson helpdesk with a computerized call waiting system for holding a limited number of incoming calls.



This solution uses three semaphores: *customers*, which counts waiting customers (excluding the customer in the barber chair, who is not waiting), *barbers*, the number of barbers (0 or 1) who are idle, waiting for customers, and *mutex*, which is used for mutual exclusion. We also need a variable, *waiting*, which also counts the waiting customers. It is essentially a copy of *customers*. The reason for having *waiting* is that there is no way to read the current value of a semaphore. In this solution, a customer entering the shop has to count the number of waiting customers. If it is less than the number of chairs, he stays; otherwise, he leaves.

Our solution is shown in Fig. below. When the barber shows up for work in the morning, he executes the procedure *barber*, causing him to block on the semaphore *customers* because it is initially 0. The barber then goes to sleep. He stays asleep until the first customer shows up.

```
#define CHAIRS 5 /* # chairs for waiting customers */  
typedef int semaphore; /* use your imagination */  
semaphore customers = 0; /* # of customers waiting for service */  
semaphore barbers = 0; /* # of barbers waiting for customers */  
semaphore mutex = 1; /* for mutual exclusion */  
int waiting = 0; /* customer are waiting (not being cut) */
```

```
void barber(void)  
{  
    while (TRUE) {  
        down(&customers); /* go to sleep if # of customers is 0 */  
        down(&mutex); /* acquire access to 'waiting' */  
        waiting = waiting - 1; /* decrement count of waiting customers */  
        up(&barbers); /* one barber is now ready to cut hair */  
        up(&mutex); /* release 'waiting' */  
        cut_hair(); /* cut hair (outside critical region) */  
    }  
}
```

```
void customer(void)  
{  
    down(&mutex); /* enter critical region */  
    if (waiting < CHAIRS) { /* if there are no free chairs, leave */  
        waiting = waiting + 1; /* increment count of waiting customers */  
    }
```

```
up(&customers); /* wake up barber if necessary */
up(&mutex); /* release access to 'waiting' */
down(&barbers); /* go to sleep if # of free barbers is 0 */
get_haircut(); /* be seated and be served */
} else {
up(&mutex); /* shop is full; do not wait */
}
}
```

When a customer arrives, he executes *customer*, starting by acquiring *mutex* to enter a critical region. If another customer enters shortly thereafter, the second one will not be able to do anything until the first one has released *mutex*. The customer then checks to see if the number of waiting customers is less than the number of chairs. If not, he releases *mutex* and leaves without a haircut.

If there is an available chair, the customer increments the integer variable, *waiting*. Then he does an up on the semaphore *customers*, thus waking up the barber. At this point, the customer and barber are both awake. When the customer releases *mutex*, the barber grabs it, does some housekeeping, and begins the haircut.

When the haircut is over, the customer exits the procedure and leaves the shop. The barber loops, however, to try to get the next customer. If one is present, another haircut is given. If not, the barber goes to sleep.